DBMS

Unit-1

Database System Applications: File Systems versus a DBMS, Database Languages, the Data Model,
Levels of Abstraction in a DBMS, Data Independence, Structure of a DBMS, Introduction to DBA, Roles
of DBA.
Introduction to Database Design: Database Design and ER Diagrams, Entities, Attributes, and Entity
Sets, Relationships and Relationship Sets, Additional Features of the ER Model, Conceptual Design With
the ER Model.

- **database** is a collection of data, typically describing the activities of one or more related organizations.
- **File system:** "A file system is a software system that manages and controls the data files in a computer system."
- **DBMS:** "A database management system (DBMS) is software used for creating and managing databases; it provides a systematic way to access, update, and delete data."

Difference: File System vs DBMS

S.No	File System	Database Management System (DBMS)
1	Manages and controls data files in a computer system.	Creates and manages databases; systematic access, update, delete.
2	Does **not** support multi-user access.	Supports **multi-user access** (concurrency).
3	**Low data consistency**.	**High data consistency** (normalization, constraints).
4	Not secured.	**Highly secured** (authentication, authorization).
5	Stores **unstructured** data.	Stores **structured** data.
6	**High data redundancy**.	**Low data redundancy**.

7	No built-in backup & recovery.	Provides **backup & recovery** mechanisms.
8	Easy to handle.	More complex to handle (but powerful).
9	Low cost.	Higher cost (software, admin).
10	Application failure doesn't affect others.	DB failure affects all dependent applications.
11	Data scattered in files → **no easy sharing**.	Centralized → **data sharing possible**.
12	No concurrency control.	Provides **concurrency control** (transactions).
13	Examples: NTFS, EXT.	Examples: Oracle, MySQL, SQL Server.

ADVANTAGES OF DBMS

1. **Data Independence** → hides storage details (abstract view).
2. **Efficient Data Access** → optimized techniques for retrieval/storage.
3. **Data Integrity & Security** → constraints + access control (e.g., salary vs budget).
4. **Data Administration** → centralized control when multiple users share data.
5. **Concurrent Access & Crash Recovery** → schedules multiple users safely + protects from failures.
6. **Reduced Application Development Time** → high-level interface = faster development.

DATA MODEL

- A **data model** = collection of **high-level data description constructs** that **hide low-level storage details**.
- A **DBMS** allows users to define how data is stored using a **data model**.
- **Semantic data models** are high-level, abstract, and help describe enterprise data.
- **ER Model** = most widely used semantic data model (pictorial).

Types of Data Models

1. **Relational Model**
    - Central concept = **Relation (table)** = set of records.
    - Schema defines → relation name, attribute names, types.

- Example:

  Students (sid: string, name: string, login: string, age: integer, gpa: real)
-

- Most DBMS today use relational model.

2. **Entity–Relationship (ER) Model**
   - **High-level pictorial model**.
   - Used for **database design**.
   - Easy for both stakeholders & developers.
   - ER Diagram = visual tool for entities, attributes, relationships.
3. **Object-Oriented Data Model**
   - Data + relationships stored together as **objects**.
   - Can store **audio, video, images** (multimedia).
   - Objects connected via **links**.
   - Closer to real-world representation.
4. **Hierarchical Model**
   - **First DBMS model**.
   - Data stored in **tree structure** (root → children).
   - Good for hierarchical data like recipes, website sitemap.
5. **Network Model**
   - Extension of hierarchical model.
   - Difference: **records can have more than one parent**.
   - Tree replaced with **graph structure**.
   - Very popular before relational model.

Levels of Data Abstraction (3-Level Architecture)

👉 It hides details of how data is stored/managed, showing only what each user needs.

1. Physical Level

- Lowest level.
- Describes **how data is stored** (actual storage details, data structures, indexes, records).
- Example: how a customer record is stored on disk blocks.

2. Logical Level

- Middle level.
- Describes **what data is stored** and **relationships among data**.

- Example schema:

type customer = record
- customer_id : string;
- customer_name : string;
- customer_street : string;
- customer_city : string;
- end;
-

3. View Level

- Highest level (user's view).
- **Application programs** or users see only part of the data.
- Used for **security** (hides sensitive info like salary).
- Example: HR staff can see employee_name but not employee_salary.

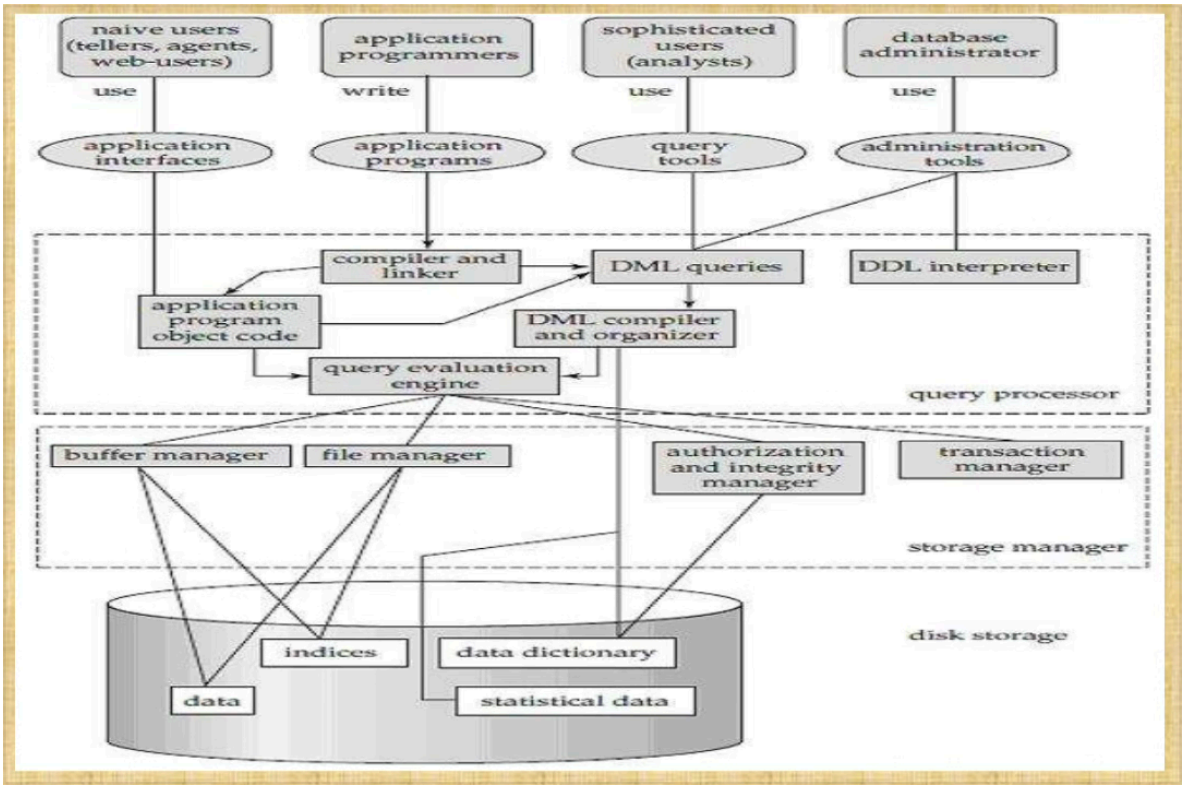
Data Independence

- **Definition:**
  The ability to modify the schema at one level **without affecting** the schema at the next higher level.
- Purpose: gives **flexibility** and reduces dependency between data storage, structure, and applications.

Types of Data Independence

1. **Logical Data Independence**
   - Ability to modify **logical schema** without affecting the **external schema (user views)**.
   - Example: Adding a new attribute email in the student table should not force changes in user applications that don't use email.
2. **Physical Data Independence**
   - Ability to modify the **physical schema** without changing the **logical schema**.
   - Example: Changing how data is stored on disk (e.g., using indexing, different file structures) should not affect the tables/ relations defined at the logical level.

Structure of DBMS



A DBMS is divided into **modules** to handle different responsibilities.
Two main functional components:

1. **Query Processor**
2. **Storage Manager**

1. Query Processor

- Interprets **user queries** and converts them into **low-level instructions**.
- Ensures queries are executed efficiently.

Components:

- **DML Compiler** → Converts DML (Data Manipulation Language) statements into low-level instructions.
- **DDL Interpreter** → Processes DDL (Data Definition Language) statements, updates metadata.
- **Embedded DML Pre-compiler** → Converts DML statements inside application programs into procedural calls.
- **Query Optimizer** → Chooses most efficient execution strategy.

2. Storage Manager

- Provides an interface between **data stored in database** and **queries**.
- Also called **Database Control System**.
- Maintains **consistency** and **integrity** of the database.
- Executes **DCL (Data Control Language)** statements.
- Handles storing, updating, deleting, retrieving data.

Components:

- **Authorization Manager** → Ensures role-based access (security).

- **Integrity Manager** → Enforces integrity constraints.
- **Transaction Manager** → Controls concurrent access, ensures consistency before & after transactions.
- **File Manager** → Manages space allocation and data structures.
- **Buffer Manager** → Manages cache memory, transfers data between main & secondary memory.

3. Disk Storage

- **Data Files** → Stores the actual data.
- **Data Dictionary** → Stores metadata (information about structure of DB objects).
- **Indices** → Provide faster retrieval of data.

Types of Database Users

1. Database Administrator (DBA)

- Super-user, responsible for **managing overall DBMS**.
- Handles **all three levels** (Physical, Logical, View).
- **Responsibilities:**
  - Defining schema & instances.
  - User liaison & support.
  - **Security** → authentication, access control.
  - **Backup & Recovery** → industry standard procedures.
  - **Performance Monitoring** → tuning database, adding resources.
  - **Software Installation & Configuration** → Oracle, SQL Server, etc.
  - **ETL (Extraction, Transformation, Loading)** → importing external data into data warehouse.
  - **Data Handling** → manage structured + unstructured (images, videos).

2. Database Designers

- Decide **database structure & schema**.
- Ensure proper **data sharing**.

3. System Analysts

- Analyze **end-user requirements**.
- Especially focus on **naïve/parametric users**.

4. Application Programmers

- Write application programs using DBMS.
- Example: developers of railway booking system, banking software.

5. Naïve / Parametric Users

- **Unskilled users** with no DBMS knowledge.
- Use **predefined applications** only.
- Examples:
  - Railway ticket booking users.
  - Clerical staff in banks.

6. Sophisticated Users
- Engineers, scientists, business analysts.
- Familiar with database concepts.
- Interact with DBMS **without writing full programs**.

7. Casual / Temporary Users
- Access database **occasionally**.
- Not regular users; short-term use only.

Participation in ER Model
Participation tells us **whether all or only some entities** of an entity set are involved in a relationship set.
In simple words → It specifies **how many entities take part** in a relationship.

1. Total Participation (Mandatory Participation)
- Every entity in the entity set **must participate** in at least one relationship.
- Represented by a **double line** between entity set and relationship set.
- **Example:**
  - Entity: Student
  - Relationship: Enrolled in
  - Meaning: Every student **must be enrolled** in at least one course.

2. Partial Participation (Optional Participation)
- An entity in the entity set **may or may not participate** in the relationship.
- Represented by a **single line** between entity set and relationship set.
- **Example:**
  - Entity: Course
  - Relationship: Enrolled in
  - Meaning: Some courses **might have no students enrolled**.

✅ **Difference Table**

Feature	Total Participation	Partial Participation
Meaning	Every entity must participate in the relationship	Entities may or may not participate
Compulsory?	Yes (Mandatory)	No (Optional)

Notation in ER diagram	Double line	Single line
Example	Every Student must enroll in a Course	A Course may have 0 students

Additional Features of ER Model

1. Generalization

- **Definition:** Process of **combining multiple lower-level entity sets** into a **single higher-level entity set**.
- Represented with **ISA hierarchy** (superclass–subclass).
- Example:
  - Entities: Car, Truck
  - Generalized Entity: Vehicle.
- Think of it as **bottom-up approach**.

2. Specialization

- **Definition:** Opposite of generalization.
- **Dividing an entity set into subgroups** based on distinguishing characteristics.
- Also represented with **ISA hierarchy**.
- Example:
  - Entity: Employee
  - Specialized Entities: Manager, Technician.
- Think of it as **top-down approach**.

3. Aggregation

- **Problem:** ER model cannot directly show **relationships between relationships**.
- **Solution:** Treat a **relationship set as a higher-level entity**.
- Example:
  - Relationship: Works_In(Employee, Department)
  - Another relationship: Manages(Project, Works_In)
  - Here, Works_In is treated as an entity.

Conceptual Design Decisions

(a) Entity vs Attribute

- Should something be modeled as an **entity** or **attribute**?
- **Rule:**
  - If it has **multiple values** → Entity (e.g., Employee can have multiple Addresses).
  - If it has **structure** → Entity (e.g., Address = City + Street + Zip).
  - Otherwise, simple **attribute** is enough.

(b) Entity vs Relationship

- If association has its **own meaning/attributes**, make it a **relationship**.
- Example:
  - Manages(Employee, Department, dbudget) → dbudget linked to Manager–Dept relationship.
  - Wrong to keep dbudget inside Department (redundant & misleading).

(c) Binary vs Ternary Relationship

- Sometimes two binary relationships are enough.
- But if the association **involves 3 entities together**, use **ternary**.
- Example:
  - Contracts(Part, Supplier, Department, qty) = ternary relationship.
  - Cannot be captured correctly by combining binary relationships.

(d) Aggregation vs Ternary

- Use **Aggregation** when a **relationship itself participates** in another relationship.
- Use **Ternary** when **three entities are directly connected**.

Database Design and ER Diagrams

The **database design process** has **six main steps**.
👉 The **ER model** is most relevant to the **first three steps**.

1. Requirements Analysis

- **Goal:** Understand what data must be stored and what applications will use it.
- Also identify:
  - Most frequent operations
  - Performance requirements

2. Conceptual Database Design

- Convert requirements into a **high-level description** of data.
- Use **ER model** to capture:
  - Entities
  - Relationships
  - Constraints

3. Logical Database Design

- Choose the **DBMS**.
- Convert the **ER diagram** into a **schema** in the DBMS's data model.
- Example: Convert ER diagram into **Relational Schema (tables)** if using relational DBMS.

Beyond ER Design

4. Schema Refinement
- Analyze the relational schema.
- Identify and fix problems like **redundancy** and **anomalies** (using normalization).

5. Physical Database Design
- Consider **workload & performance**.
- Decide:
  - Indexes
  - Storage structures
  - File organization

6. Application and Security Design
- Design **applications** and **user interfaces**.
- Define **security policies** (who can access what).

Entities, Attributes, and Entity Sets

1. Entities
- An **entity** is a real-world object that exists and is distinguishable.
- **Examples:**
  - Person → STUDENT, PROFESSOR
  - Place → UNIVERSITY, STORE
  - Object → MACHINE, BUILDING
  - Event → SALE, REGISTRATION
- **Notation:** Rectangle (Double rectangle = Weak Entity).

2. Attributes
- Properties that describe an entity or relationship.
- **Notation:** Ellipse (Double ellipse = Multivalued, Dashed ellipse = Derived).
- **Types of Attributes:**
  - **Simple:** Cannot be divided further (e.g., Age).
  - **Composite:** Can be divided into sub-parts (e.g., Address → Street, City, Pin).
  - **Derived:** Value derived from another attribute (e.g., Age from DOB).
  - **Single-Valued:** Holds one value only (e.g., Roll Number).
  - **Multi-Valued:** Can hold multiple values (e.g., Phone Numbers).

3. Entity Sets
- A collection of **entities of the same type**.
- Example: All students = STUDENT entity set.

4. Relationships

- Association among entities.
- **Notation:** Diamond (Double diamond = Identifying Relationship for weak entities).
- **Relationship Set:** Collection of similar relationships.

Types:

- **Binary (between two entities)**
  - One-to-One (1:1) → Rare. Example: 1 Student ↔ 1 ID Card.
  - One-to-Many (1:N) → Example: 1 Department ↔ Many Students.
  - Many-to-One (N:1) → Example: Many Students ↔ 1 Course.
  - Many-to-Many (M:N) → Example: Many Students ↔ Many Courses.
- **Recursive Relationship** → Entity related to itself. Example: Employee manages Employee.
- **Ternary Relationship** → Involves 3 entities. Example: Supplier – Supplies – Part – Department.

5. Strong Entity

- Independent, has **primary key**.
- Notation: **Single Rectangle**.
- Relationship with another strong entity → **Single Diamond**.

6. Weak Entity

- Dependent on **Strong Entity**.
- Identified using a **Partial Key** + Primary Key of owner strong entity.
- Notation: **Double Rectangle**.
- Relationship with strong entity = **Double Diamond (Identifying Relationship**